CPEN 416

Lecture 12: Quantum compilation II (continued); the quantum Fourier transform (QFT)

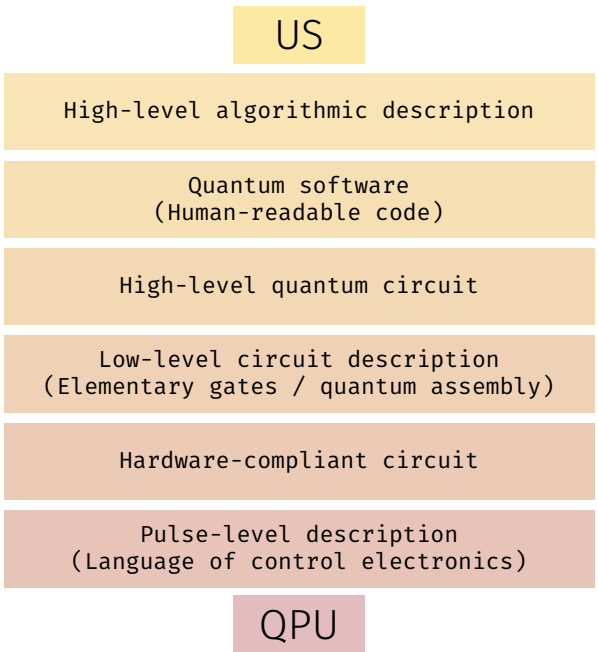
Thursday 22 October 2026

Announcements

- Monday's tutorial: Tutorial Assignment 2
- Quiz 6 Tuesday
- Final project group selection due by 5pm today
- Tomorrow's office hour on Zoom (link in Canvas)
- Final exam scheduled Wednesday 16 December 15:30-17:30

Last time

We were working our way down the quantum compilation stack.



Implemented as a sequence of modular **passes**, or **transforms**, that modify a circuit in various ways.

Loosely divided into 3 stages:

- decomposition and synthesis
- hardware-independent optimization
- hardware-dependent optimization

Today

Learning outcomes:

- Compile and optimize Grover circuits to a topology-restricted quantum processor
- Express floating-point values in fractional binary representation
- Describe the behaviour of the quantum Fourier transform
- Construct a circuit for the quantum Fourier transform and analyze its resource usage

Qubit placement and routing

US

High-level algorithmic description

Quantum software
(Human-readable code)

High-level quantum circuit

Low-level circuit description
(Elementary gates / quantum assembly)

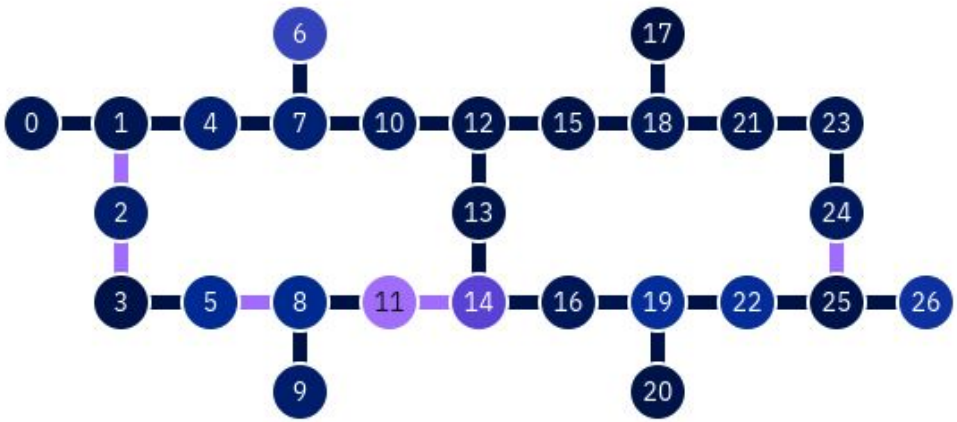
Hardware-compliant circuit

Pulse-level description
(Language of control electronics)

QPU

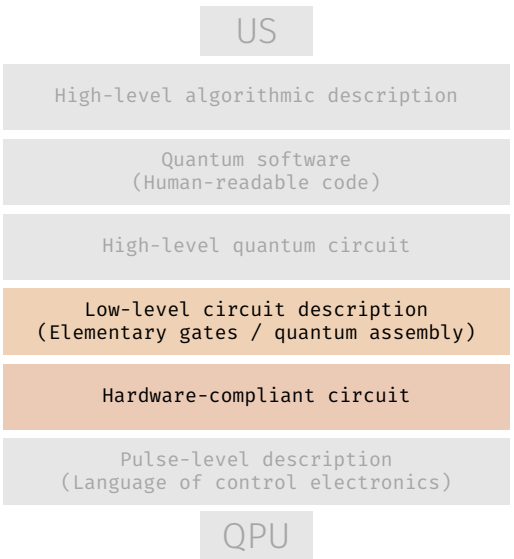
Each type of machine has its own compilation challenges:

- superconducting machines have restricted topology



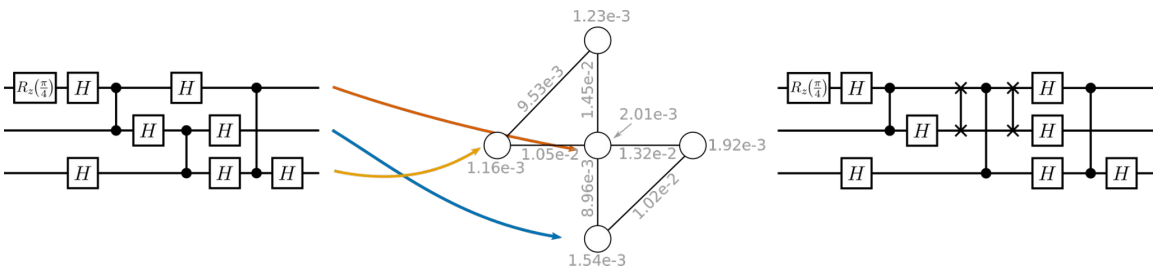
IBM Q Kolkata — screencap 2023-08-24

Qubit placement and routing



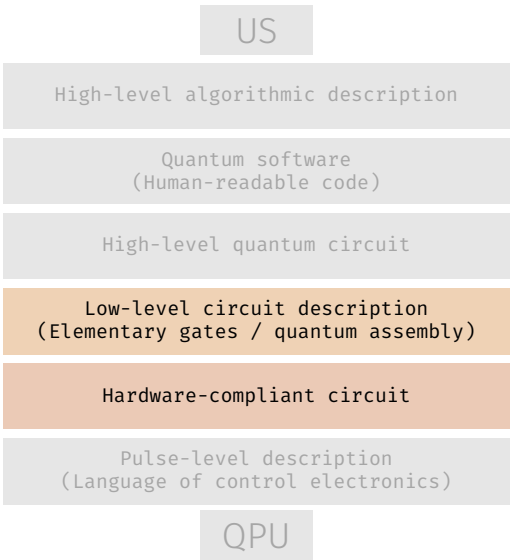
Each type of machine has its own compilation challenges:

- superconducting machines have restricted topology



Placement is NP-complete; routing is NP-hard.

Qubit placement and routing



Each type of machine has its own compilation challenges:

- superconducting machines have restricted topology
- neutral atom machines can lose atoms

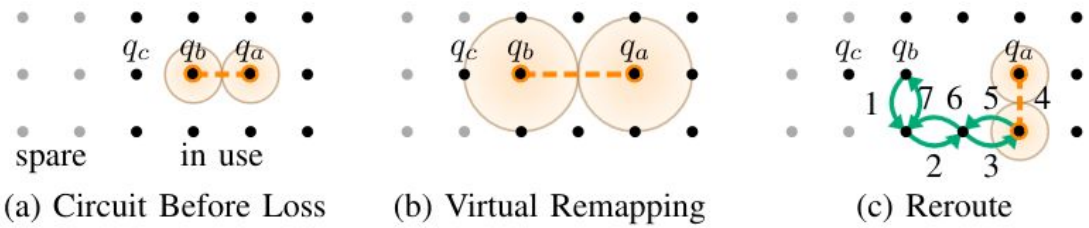


Image credit: Baker et al. (2021) *Exploiting Long-Distance Interactions and Tolerating Atom Loss in Neutral Atom Quantum Architectures*. ISCA '21.

Qubit placement and routing

US

High-level algorithmic description

Quantum software
(Human-readable code)

High-level quantum circuit

Low-level circuit description
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description
(Language of control electronics)

QPU

Techniques can use optimal solvers, heuristics, or a combination of both.

Example: SABRE ("SWAP-based BidiREctional heuristic search")

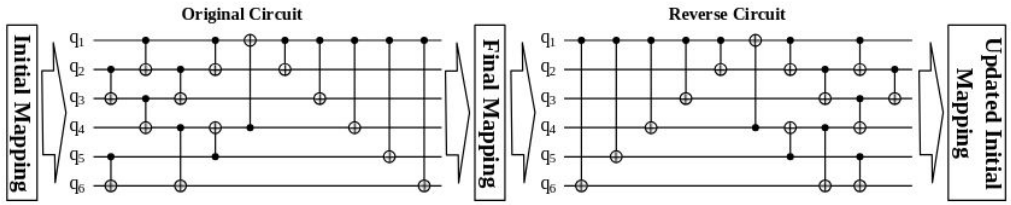


Fig. 5: Initial Mapping Update Using Reverse Traversal Technique

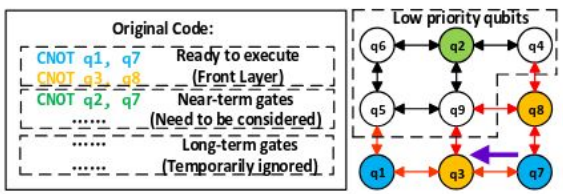
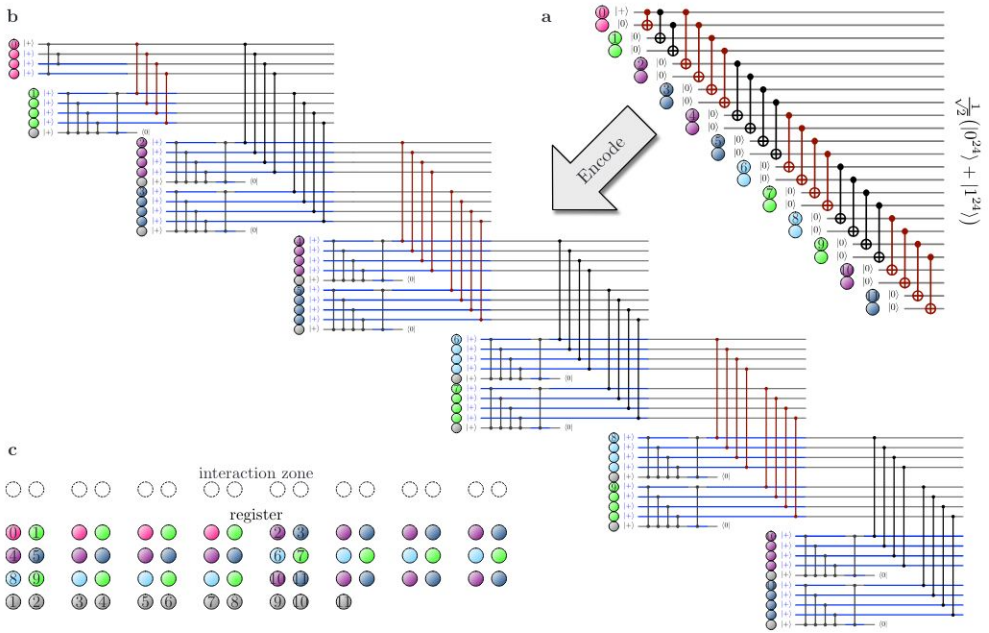


Fig. 6: Example of SWAP-Based Heuristic Search

Image credit: G. Li, Y. Ding, and Y. Xie (2019) *Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices*. ASPLOS '19, 1001–1014.

Gate scheduling



Recent demonstration of logical qubits on neutral atom machine: *move qubits in and out of "interaction zone" for 2-qubit gates*

(Microsoft + Atom Computing + collaborators)

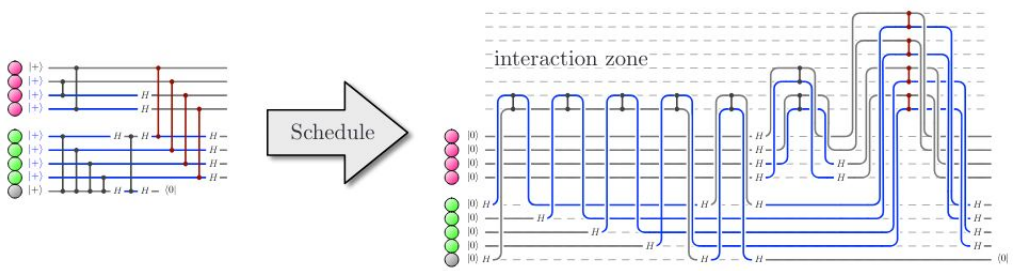


Image credit: Fig. 4 and 5, B. W. Reichardt et al., *Logical computation demonstrated with a neutral atom quantum processor*. arXiv:2411.11822 [quant-ph].

Pulse design and optimization

US

High-level algorithmic description

Quantum software
(Human-readable code)

High-level quantum circuit

Low-level circuit description
(Elementary gates / quantum assembly)

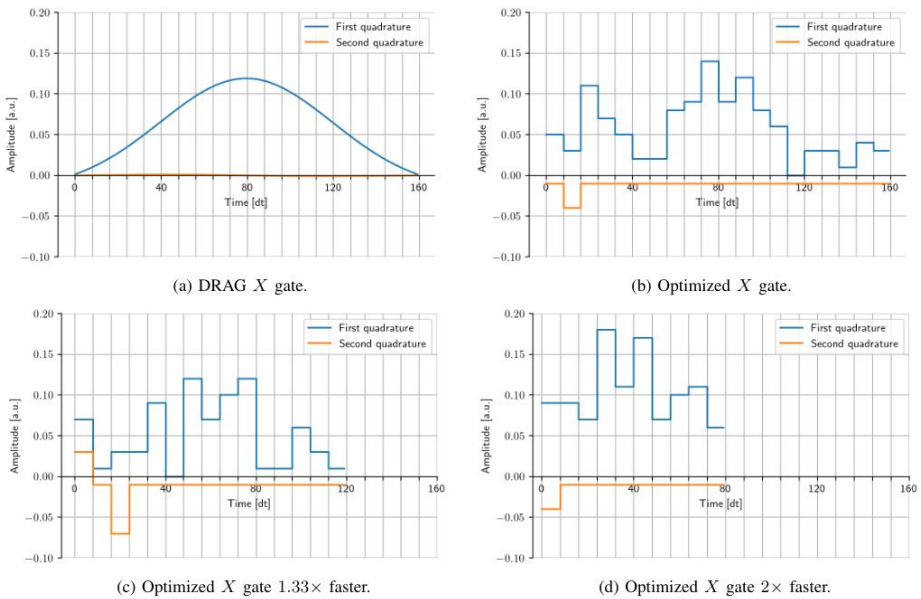
Hardware-compliant circuit

Pulse-level description
(Language of control electronics)

QPU

Electromagnetic pulses are what *actually* implement the gates in a circuit. Interesting problems here:

- optimizing duration/shape



(a) DRAG X gate. (b) Optimized X gate.

(c) Optimized X gate 1.33× faster. (d) Optimized X gate 2× faster.

Image credit: E. Wright and R. de Sousa (2023) *Fast quantum gate design with deep reinforcement learning using real-time feedback on readout signals*. arXiv:2305.01169 [quant-ph]

Pulse design and optimization

US

High-level algorithmic description

Quantum software
(Human-readable code)

High-level quantum circuit

Low-level circuit description
(Elementary gates / quantum assembly)

Hardware-compliant circuit

Pulse-level description
(Language of control electronics)

QPU

Electromagnetic pulses are what *actually* implement the gates in a circuit. Interesting problems here:

- optimizing duration/shape
- scheduling constraints

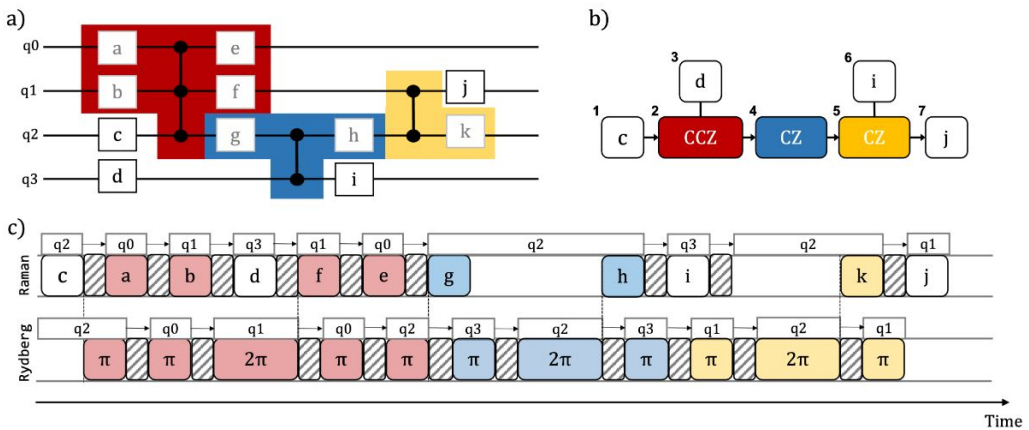


Image credit: R. B.-S. Tsai, H. Silvério, L. Henriet (2022) *Pulse-Level Scheduling of Quantum Circuits for Neutral-Atom Devices*. Phys. Rev. Applied 18 064035.

Software tools

Many general-purpose packages contain preset passes, and the ability to create your own.

```
qiskit_circuit = QuantumCircuit(...)
transpiled_circuit = transpile(qiskit_circuit, optimization_level=3,
                               coupling_map=..., layout_method="sabre")
```

```
@qp.qnode(dev)
@expand_rot_and_remove_zeros
@qp.compile(pipeline=[
    qp.transforms.cancel_inverses,
    qp.transforms.single_qubit_fusion(),
    qp.transforms.commute_controlled(direction="left"),
    qp.transforms.merge_rotations()
], num_passes=3)
def tapered_circuit_simplified(params):
    for wire in dev.wires:
        if hf_state_tapered[wire] == 1:
            qp.PauliX(wires=wire)
```

See also: Cirq, TKET, staq, XACC, and more.

Let's try it!

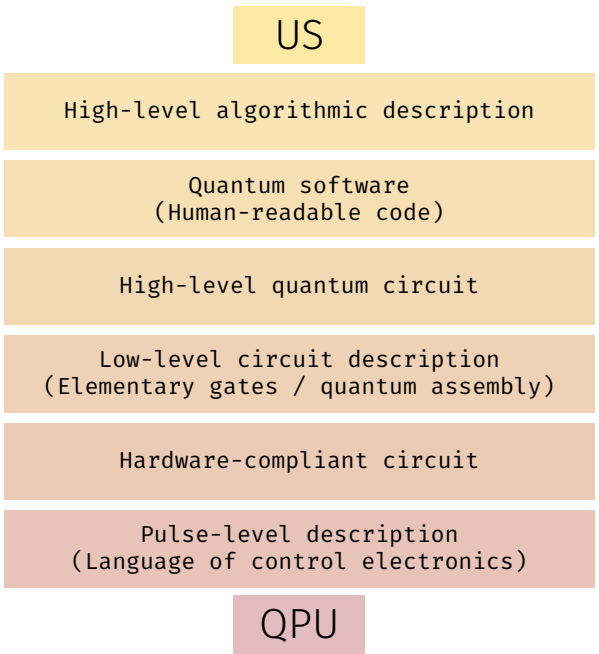
Challenge

Many things are (computationally) **hard**.

Many parts of the stack **still require expert input** and tailored design.

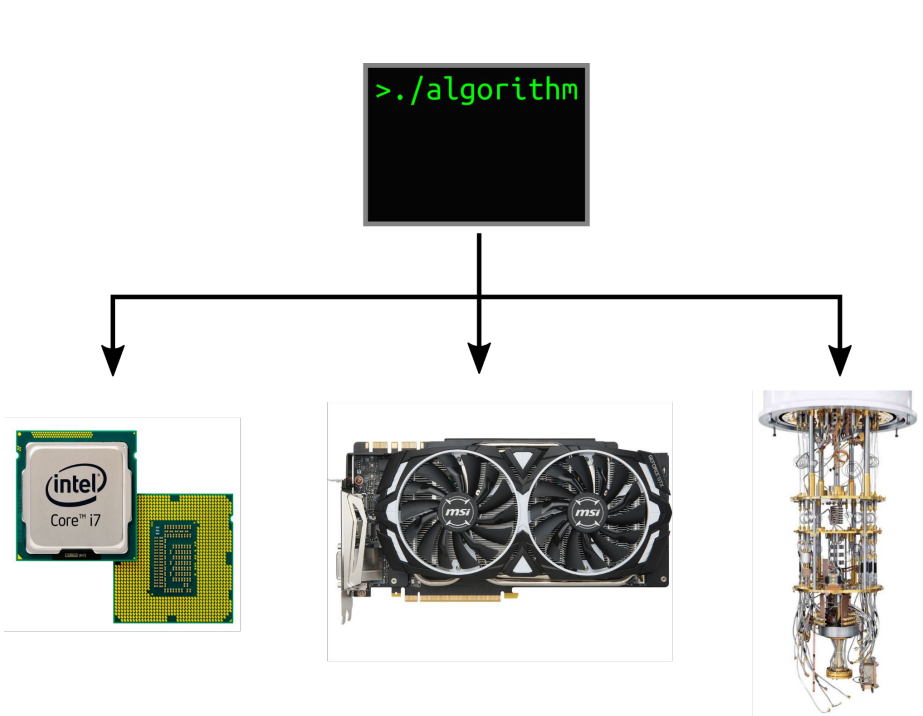
Need **automation** to scale up to (multi-QPU) devices with 1000s of qubits:

- circuit synthesis
- circuit optimization
- mapping, routing, and scheduling
- noise-aware techniques
- verification and debugging



Compiling classical and quantum code

Quantum algorithms don't exist in a vacuum.



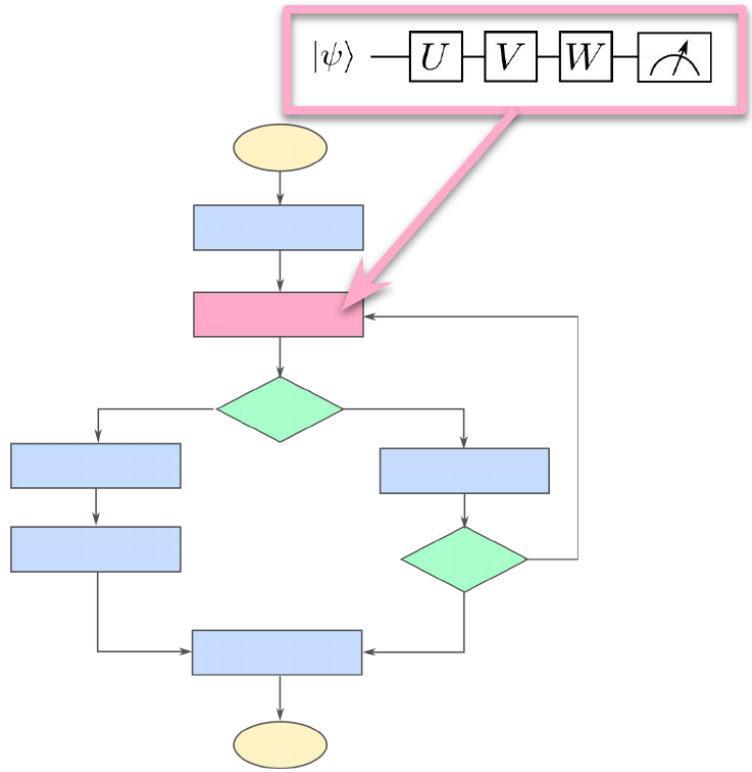
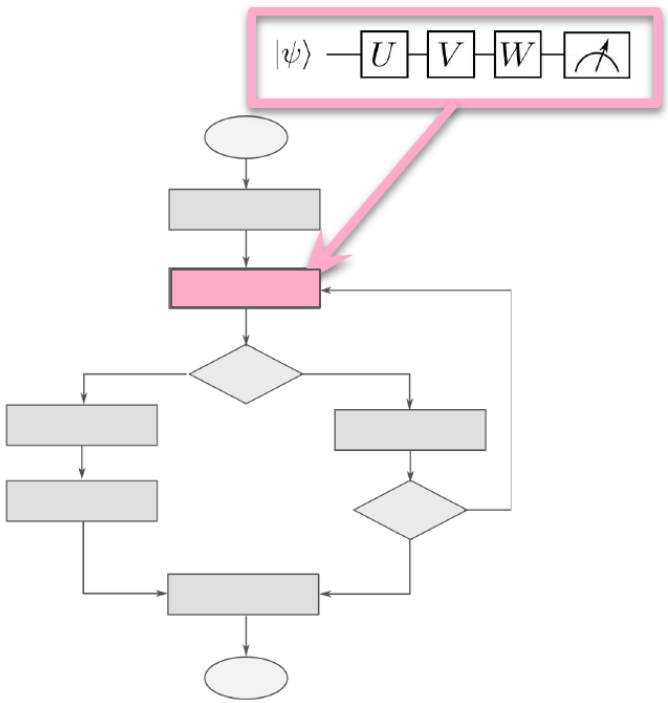


Image credits: https://phys.org/news/2012-04-intel-batch-ivy-bridge-processors.html;

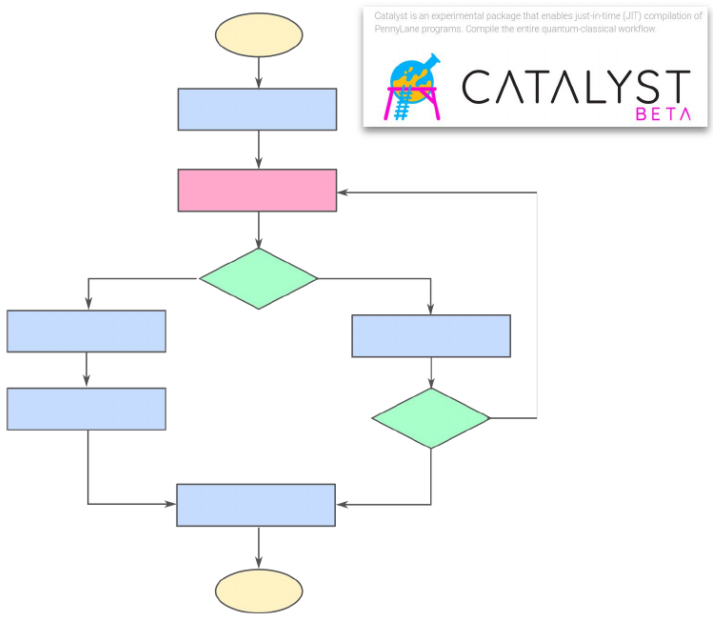
https://www.techpowerup.com/img/17-03-28/65ee0f227ed0.jpg; Photo/Justin Fantl, Rigetti Computing

Compiling classical and quantum code

So far, we've addressed the problem of compiling this part:



Can we compile the whole thing?



https://docs.pennylane.ai/projects/catalyst/en/stable/

https://github.com/PennyLaneAI/catalyst

Where have we been?

Where are we going?

```
def shors_algorithm(N):
    p, q = 0, 0

    while p * q != N:
        a = np.random.choice(list(range(2, N - 1)))

        if np.gcd(a, N) != 1:
            p = np.gcd(a, N)
            q = N // p
            return p, q

        sample = get_sample(a, N)
        phase = fractional_binary_to_float(sample)
        candidate_order = phase_to_order(phase, N)

        if candidate_order % 2 == 0:
            square_root = (a ** (candidate_order // 2)) % N

            if square_root not in [1, N - 1]:
                p = np.gcd(square_root - 1, N)
                q = np.gcd(square_root + 1, N)

    return p, q
```

Where are we going?

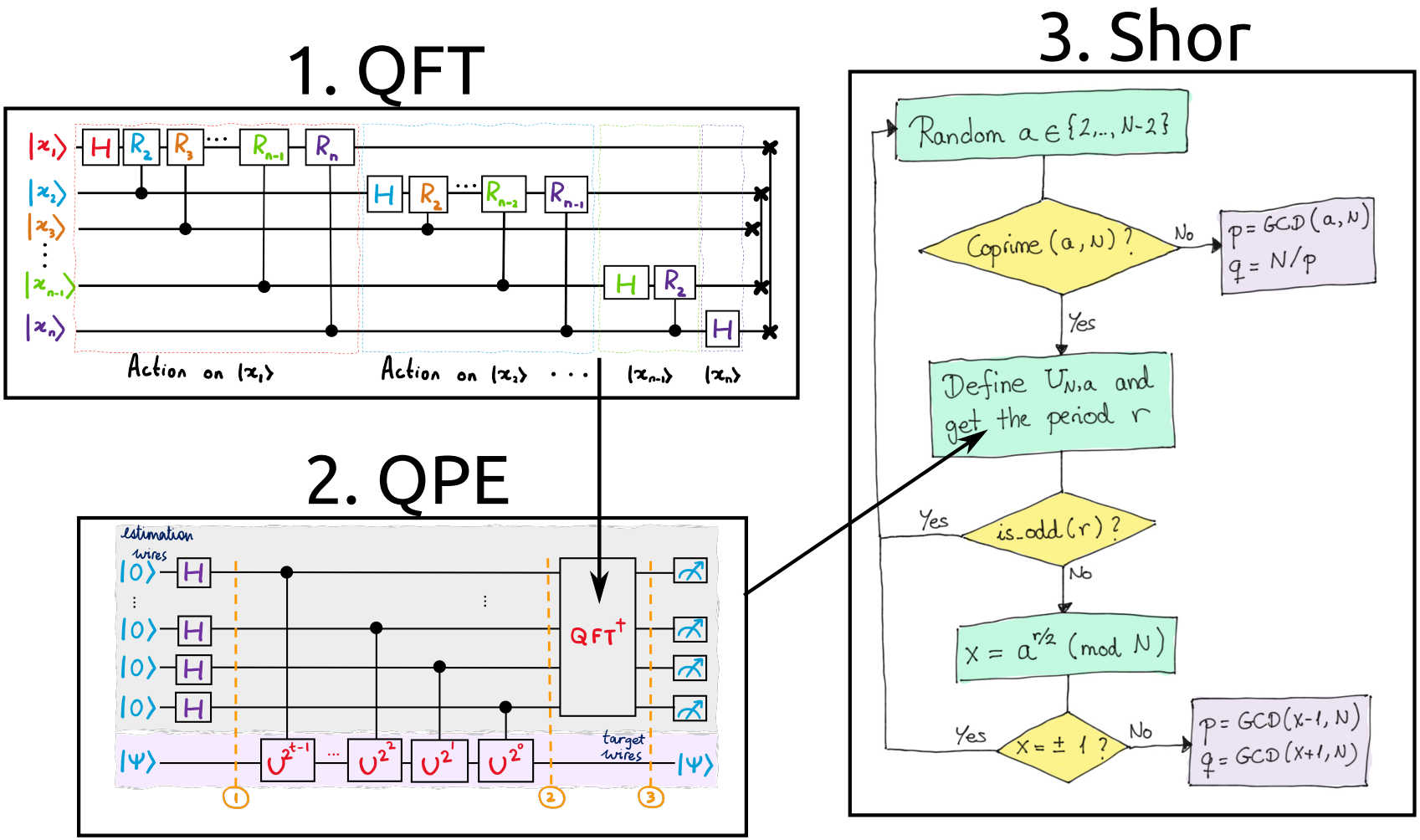


Image credits: PennyLane Codebook

Module 3 learning outcomes

Learning outcomes:

- define, and state the scaling of, the quantum Fourier transform
- use quantum phase estimation to determine the eigenvalues of a unitary matrix
- use the QFT and QPE as subroutines to implement order finding, and simulate Shor's factoring algorithm
- identify cryptographic schemes susceptible to quantum attack
- describe the societal and ethical implications of quantum technology

Fractional binary

Consider binary representation of an integer k,

We can extend this notation to floating point numbers:

Fractional binary

Exercise: let $k = 0.11010$. What is the numerical value of k?

Fractional binary

The QFT will be defined using powers of complex roots of unity:

Fractional binary representation will help us

1. derive a circuit implementation for the QFT today
2. interpret the results of *quantum phase estimation* on Thursday

The discrete Fourier transform

The DFT and FFT (which implements it efficiently) convert between time and frequency domains in digital signal processing.

$$DFT = \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \bar{\omega} & \bar{\omega}^2 & \cdots & \bar{\omega}^{N-1} \\ 1 & \bar{\omega}^2 & \bar{\omega}^4 & \cdots & \bar{\omega}^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \bar{\omega}^{N-1} & \bar{\omega}^{2(N-1)} & \cdots & \bar{\omega}^{(N-1)(N-1)} \end{pmatrix}$$

where $\bar{\omega} = e^{-2\pi i/N}$.

The discrete Fourier transform

Given a signal $x[n]$, the DFT computes

The inverse DFT computes

where $\omega = e^{2\pi i/N} = \bar{\omega}^{-1}$

Quantum Fourier transform

The quantum Fourier transform (QFT) is the quantum analog of the **inverse DFT**.

Exercise: Apply the QFT to an n-qubit basis state $|x\rangle$

Quantum Fourier transform: 1 qubit

Quantum Fourier transform: 2 qubits

$$QFT = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega & \omega^2 & \omega^3 \\ 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^3 & \omega^6 & \omega^9 \end{pmatrix} = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{pmatrix}$$

Re-express higher powers of $\omega = e^{\frac{2\pi i}{4}}$:

Quantum Fourier transform: 2 qubits

$$QFT = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega & \omega^2 & \omega^3 \\ 1 & \omega^2 & 1 & \omega^2 \\ 1 & \omega^3 & \omega^2 & \omega^1 \end{pmatrix}$$

How does this affect the computational basis states?

Quantum Fourier transform: 2 qubits

Quantum Fourier transform: 2 qubits

Quantum Fourier transform: 3 qubits

$$QFT = \frac{1}{\sqrt{8}} \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & \cdots \\ 1 & \omega & \omega^2 & \omega^3 & \omega^4 & \cdots \\ 1 & \omega^2 & \omega^4 & \omega^6 & 1 & \cdots \\ 1 & \omega^3 & \omega^6 & \omega & \omega^4 & \cdots \\ 1 & \omega^4 & 1 & \omega^4 & 1 & \cdots \end{pmatrix}$$

$$QFT|x_1x_2x_3\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i 0.x_3}|1\rangle\right) \otimes \frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i 0.x_2x_3}|1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i 0.x_1x_2x_3}|1\rangle)$$

Quantum Fourier transform: n qubits

General expression:

$$
\begin{aligned}
|x\rangle \quad &\rightarrow \quad \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega^{xk} |k\rangle \\
&= \quad \frac{\left(|0\rangle + e^{2\pi i 0.x_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.x_{n-1}x_n}|1\rangle\right) \cdots \left(|0\rangle + e^{2\pi i 0.x_1 \cdots x_n}|1\rangle\right)}{\sqrt{N}}
\end{aligned}
$$

Full mathematical derivation at end of slides.

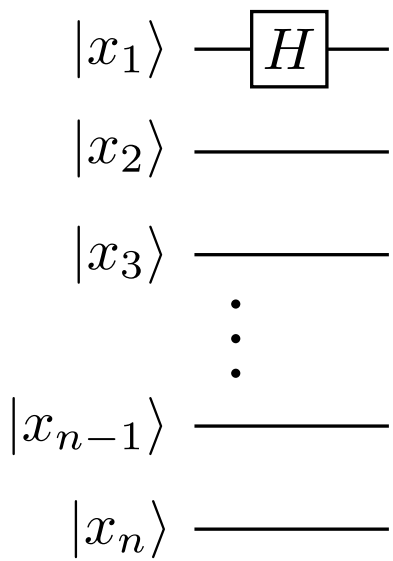
A circuit for the QFT

$$|x\rangle \rightarrow \frac{\left(|0\rangle + e^{2\pi i 0.x_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.x_{n-1}x_n}|1\rangle\right)\cdots\left(|0\rangle + e^{2\pi i 0.x_1\cdots x_n}|1\rangle\right)}{\sqrt{N}}$$

Starting with the state

$$|x\rangle = |x_1 \cdots x_n\rangle,$$

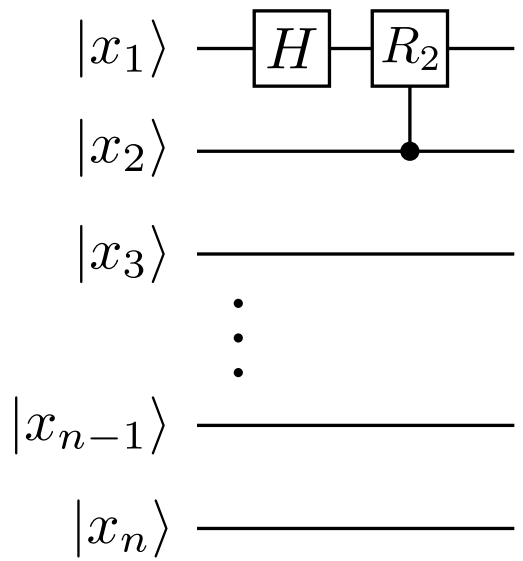
apply a Hadamard to qubit 1:



A circuit for the QFT

$$|x\rangle \rightarrow \frac{\left(|0\rangle + e^{2\pi i 0.x_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.x_{n-1}x_n}|1\rangle\right)\cdots\left(|0\rangle + e^{2\pi i 0.x_1\cdots x_n}|1\rangle\right)}{\sqrt{N}}$$

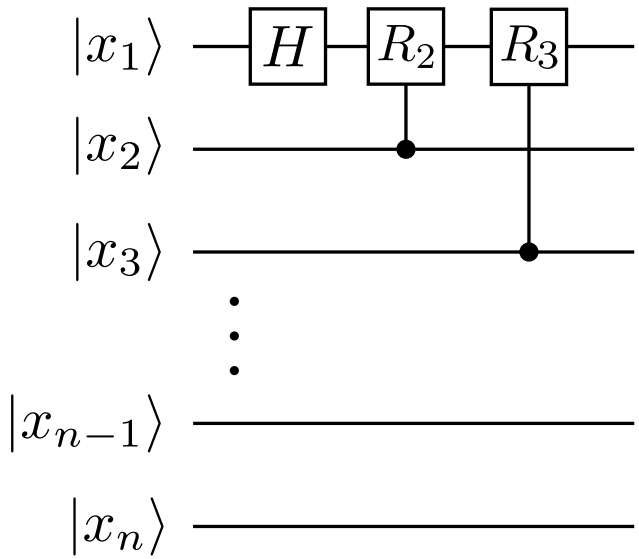
Apply controlled R_2 from qubit $2 \rightarrow 1$



A circuit for the QFT

$$|x\rangle \rightarrow \frac{\left(|0\rangle + e^{2\pi i 0.x_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.x_{n-1}x_n}|1\rangle\right)\cdots\left(|0\rangle + e^{2\pi i 0.x_1\cdots x_n}|1\rangle\right)}{\sqrt{N}}$$

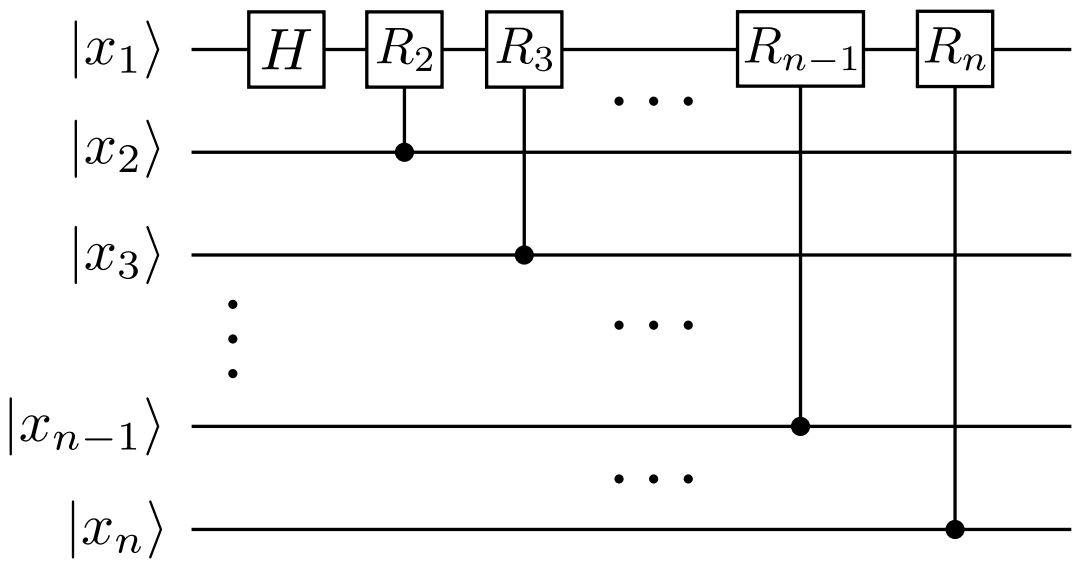
Apply controlled R_3 from qubit $3 \rightarrow 1$



A circuit for the QFT

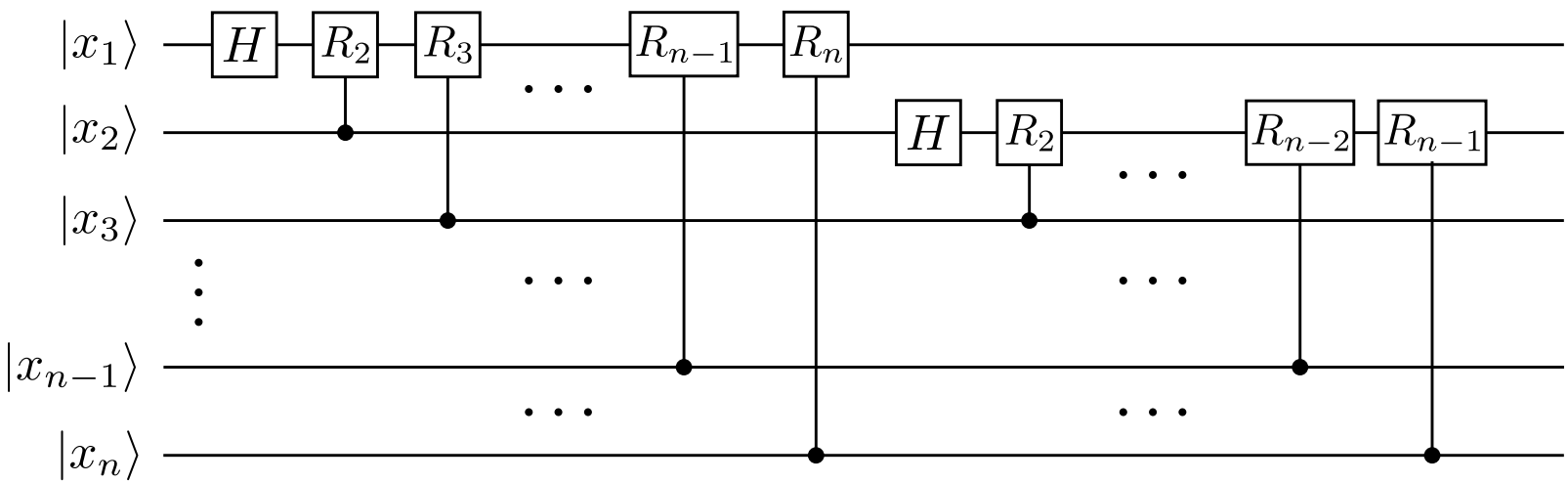
$$|x\rangle \rightarrow \frac{\left(|0\rangle + e^{2\pi i 0.x_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.x_{n-1}x_n}|1\rangle\right)\cdots\left(|0\rangle + e^{2\pi i 0.x_1\cdots x_n}|1\rangle\right)}{\sqrt{N}}$$

Apply a controlled R_4 from $4 \rightarrow 1$, etc. up to the n-th qubit to get



A circuit for the QFT

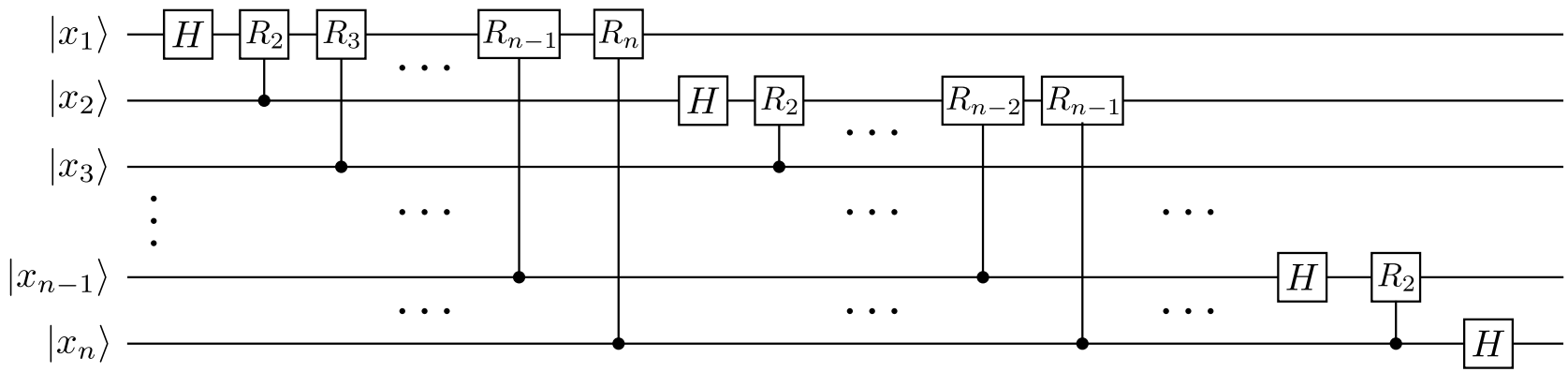
Similar thing with second qubit: apply H then controlled rotations from qubits 3 to n to get



A circuit for the QFT

Do this for all qubits to get that big ugly state from earlier:

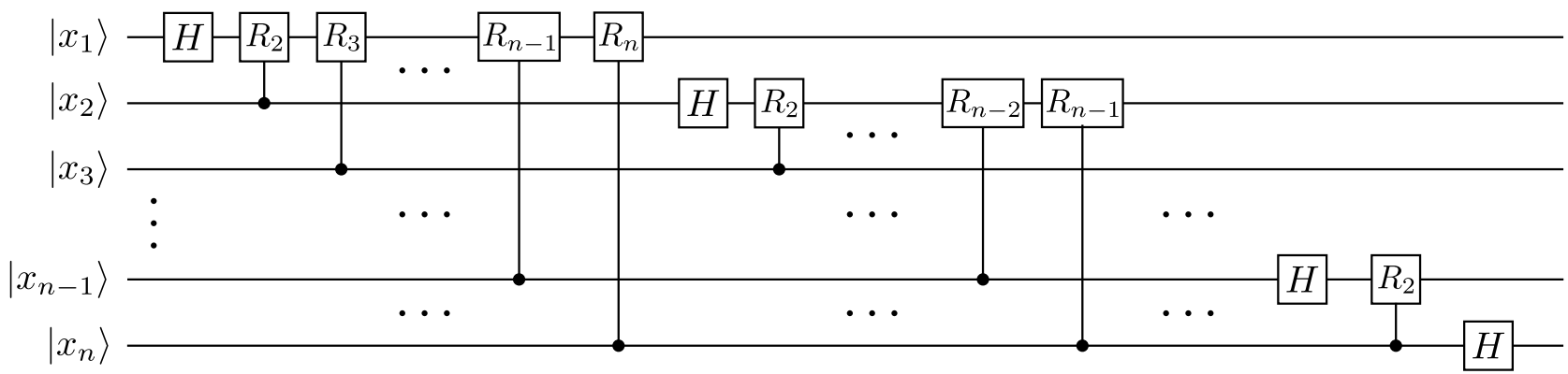
$$|x\rangle \rightarrow \frac{\left(|0\rangle + e^{2\pi i 0.x_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.x_{n-1}x_n}|1\rangle\right)\cdots\left(|0\rangle + e^{2\pi i 0.x_1\cdots x_n}|1\rangle\right)}{\sqrt{N}}$$



Qubit order is backwards - easily fixed with SWAP gates.

Quantum Fourier transform

Exercise: What are the gate counts and depth of this circuit?



Next time

Content:

- Quiz 6
- Quantum phase estimation

Action items:

1. Project group selection
2. Work on project
3. Monday's tutorial: tutorial assignment 2

Recommended reading:

- For this class: Codebook module QFT, Nielsen & Chuang 5.1
- For next class: Codebook module QPE, Nielsen & Chuang 5.2

Derivation of output state of QFT

We will reexpress k/N in fractional binary notation, then reshuffle and *factor* the output state.

$$
\begin{aligned}
|x\rangle &\rightarrow \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega^{xk} |k\rangle \\
&= \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i x(k/N)} |k\rangle \\
&= \frac{1}{\sqrt{N}} \sum_{k_1=0}^{1} \cdots \sum_{k_n=0}^{1} e^{2\pi i x\left(\sum_{\ell=1}^{n} k_\ell 2^{-\ell}\right)} |k_1 \cdots k_n\rangle \\
&= \frac{1}{\sqrt{N}} \sum_{k_1=0}^{1} \cdots \sum_{k_n=0}^{1} \bigotimes_{\ell=1}^{n} e^{2\pi i x k_\ell 2^{-\ell}} |k_\ell\rangle
\end{aligned}
$$

Derivation of output state of QFT

$$
\begin{aligned}
&= \frac{1}{\sqrt{N}} \sum_{k_1=0}^{1} \cdots \sum_{k_n=0}^{1} \bigotimes_{\ell=1}^{n} e^{2\pi i x k_\ell 2^{-\ell}} |k_\ell\rangle \\
&= \frac{1}{\sqrt{N}} \bigotimes_{\ell=1}^{n} \left(\sum_{k_\ell=0}^{1} e^{2\pi i x k_\ell 2^{-\ell}} |k_\ell\rangle \right) \\
&= \frac{1}{\sqrt{N}} \bigotimes_{\ell=1}^{n} \left(|0\rangle + e^{2\pi i x 2^{-\ell}} |1\rangle \right) \\
&= \frac{\left(|0\rangle + e^{2\pi i 0.x_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.x_{n-1}x_n}|1\rangle\right) \cdots \left(|0\rangle + e^{2\pi i 0.x_1 \cdots x_n}|1\rangle\right)}{\sqrt{N}}
\end{aligned}
$$